SEZG514 - Introduction to DEVOPS

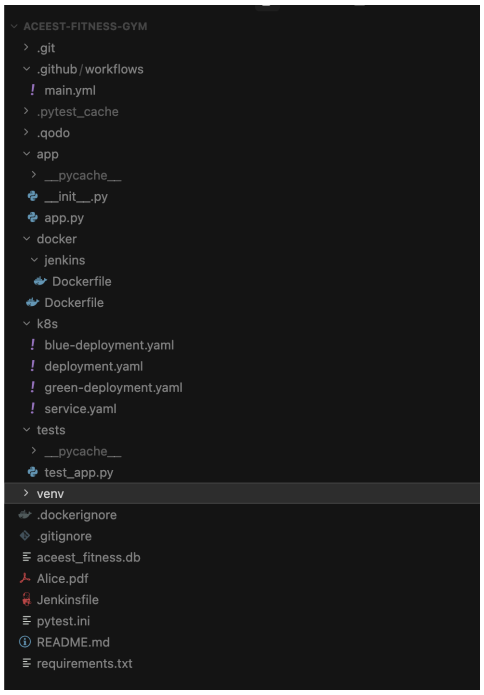
Assignment 2 - DevOps CI/CD implementation for ACEest Fitness & Gym

Submitted by: Mohit Kumar [2024TM93706]

Google cloud deployment link: http://35.200.230.182/

Github repo link: https://github.com/2024tm93706/aceest-fitness-gym

(also attached in zip)



Architecture:

Local Development → GitHub → Jenkins Pipeline → Docker → Artifact Registry → GKE

The architecture consists of:

- **Source Control**: GitHub repository hosting application code
- **CI Tool**: Jenkins pipeline for automated build and deployment
- **Containerization**: Docker for packaging the application
- **Container Registry**: Google Artifact Registry for storing images
- **Orchestration**: Google Kubernetes Engine (GKE)

- **Deployment Strategy**: Blue-Green deployment using Kubernetes services

Workflow:

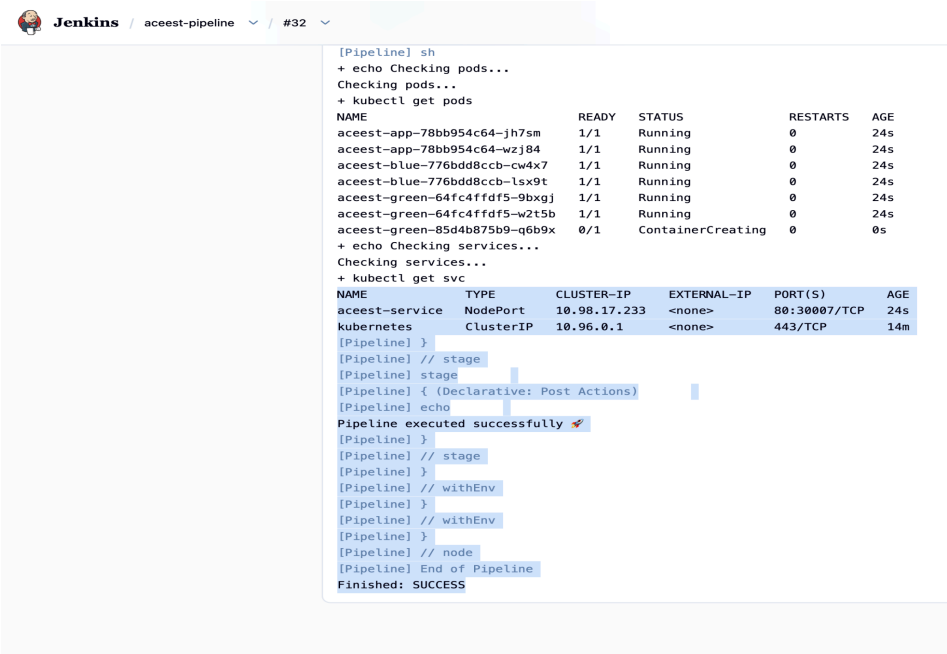
Step 1: Code Management

- Source code stored in GitHub repository
- Jenkins pulls the latest code using credentials

Step 2: Build Pipeline

- Docker image is built using a custom Dockerfile
- Multi-architecture issue resolved by forcing:
  - linux/amd64 build using Docker Buildx

Jenkins with docker and kubernetes

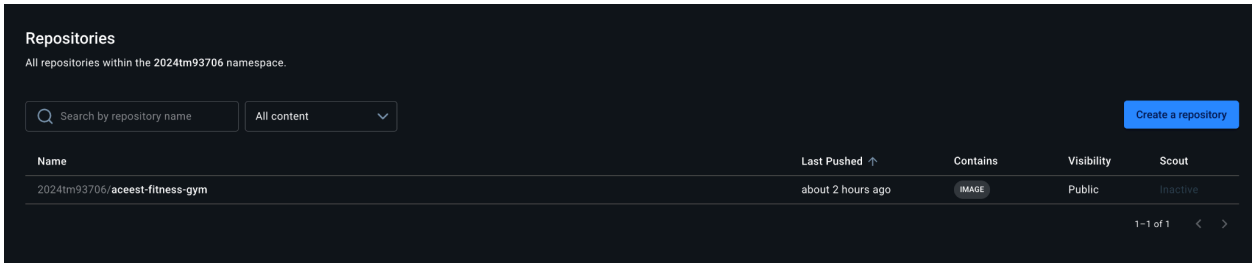


Jenkins automates the build and deployment process by triggering on code changes, building Docker images, pushing them to Artifact Registry, and updating Kubernetes deployments. This eliminates manual intervention and ensures consistent deployments.

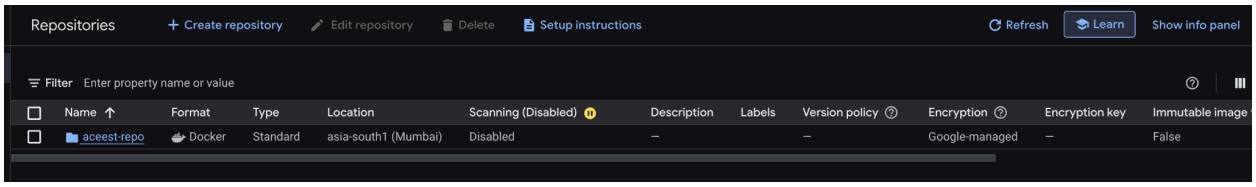
Step 3: Image Storage

- Image pushed to Google Artifact Registry:
  asia-south1-docker.pkg.dev/<project-id>/aceest-repo/app:v3

Initial docker hub repo



Later, Docker images were stored in Google Artifact Registry



Step 4: Kubernetes Deployment

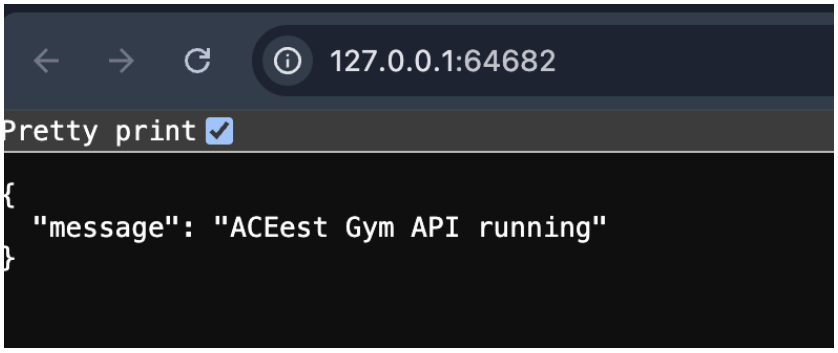
- GKE cluster created and configured
- kubectl used to deploy application

Initial deployment was performed on Minikube for local testing

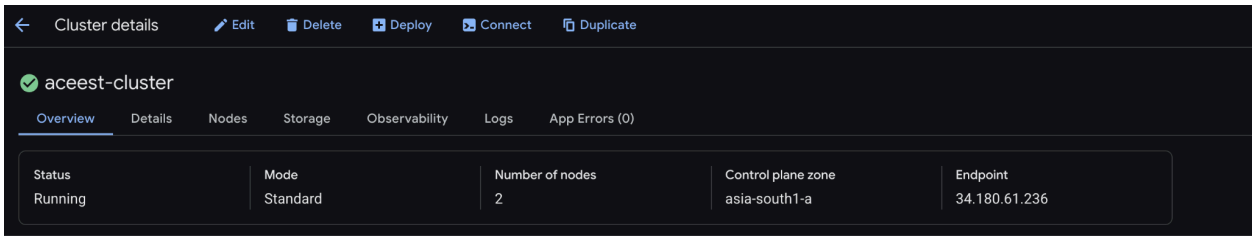
```
mkayfour@192 aceest-fitness-gym % kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
aceest-app-6b96d87d79-k6ghs       1/1     Running   0          57s
aceest-app-6b96d87d79-xq6d5       1/1     Running   0          57s
mkayfour@192 aceest-fitness-gym %
```

```
mkayfour@192 aceest-fitness-gym % kubectl get svc -A
NAMESPACE             NAME                        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                  AGE
default               kubernetes                  ClusterIP   10.96.0.1       <none>        443/TCP                  5m36s
kube-system           kube-dns                    ClusterIP   10.96.0.10      <none>        53/UDP,53/TCP,9153/TCP   5m35s
kubernetes-dashboard  dashboard-metrics-scraper   ClusterIP   10.106.225.23   <none>        8000/TCP                 4m36s
kubernetes-dashboard  kubernetes-dashboard        ClusterIP   10.111.160.63   <none>        80/TCP                   4m36s
mkayfour@192 aceest-fitness-gym %
```

```
mkayfour@192 aceest-fitness-gym % kubectl get svc -A
NAMESPACE             NAME                        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                  AGE
default               kubernetes                  ClusterIP   10.96.0.1       <none>        443/TCP                  6m11s
kube-system           kube-dns                    ClusterIP   10.96.0.10      <none>        53/UDP,53/TCP,9153/TCP   6m10s
kubernetes-dashboard  dashboard-metrics-scraper   ClusterIP   10.106.225.23   <none>        8000/TCP                 5m11s
kubernetes-dashboard  kubernetes-dashboard        ClusterIP   10.111.160.63   <none>        80/TCP                   5m11s
mkayfour@192 aceest-fitness-gym % kubectl apply -f k8s/service.yaml
service/aceest-service created
mkayfour@192 aceest-fitness-gym % kubectl get svc
NAME            TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
aceest-service  NodePort    10.109.189.210   <none>        80:30007/TCP   7s
kubernetes      ClusterIP   10.96.0.1        <none>        443/TCP        6m47s
mkayfour@192 aceest-fitness-gym % minikube service aceest-service
|-----------|----------------|-------------|---------------------------|
| NAMESPACE |      NAME      | TARGET PORT |           URL             |
|-----------|----------------|-------------|---------------------------|
| default   | aceest-service | 80          | http://192.168.49.2:30007 |
|-----------|----------------|-------------|---------------------------|
🏃  Starting tunnel for service aceest-service.
|-----------|----------------|-------------|-----------------------|
| NAMESPACE |      NAME      | TARGET PORT |          URL          |
|-----------|----------------|-------------|-----------------------|
| default   | aceest-service |             | http://127.0.0.1:64682 |
|-----------|----------------|-------------|-----------------------|
🏃  Starting tunnel for service aceest-service.
🎉  Opening service default/aceest-service in default browser...
❗  Because you are using a Docker driver on darwin, the terminal needs to be open to run it.
```



Later on Google cloud [Kubernetes engine]



Deployment Strategy: Blue-Green

Two separate deployments were created:
- aceest-blue → Stable version
- aceest-green → New version

Each deployment:
- Runs identical application instances
- Uses labels:
  - app: aceest
  - version: blue/green

```
mkayfour@192 aceest-fitness-gym % kubectl apply -f k8s/service.yaml
service/aceest-service configured
mkayfour@192 aceest-fitness-gym % kubectl get endpoints aceest-service
NAME             ENDPOINTS                                                     AGE
aceest-service   10.244.0.19:5000,10.244.0.20:5000,10.244.0.21:5000 + 1 more...   88m
mkayfour@192 aceest-fitness-gym %
```

```
mkayfour@192 aceest-fitness-gym % kubectl get endpoints aceest-service
NAME             ENDPOINTS                                                     AGE
aceest-service   10.244.0.19:5000,10.244.0.20:5000,10.244.0.21:5000 + 1 more...   88m
mkayfour@192 aceest-fitness-gym % kubectl scale deployment aceest-blue --replicas=3
deployment.apps/aceest-blue scaled
mkayfour@192 aceest-fitness-gym % kubectl scale deployment aceest-green --replicas=1
deployment.apps/aceest-green scaled
mkayfour@192 aceest-fitness-gym % kubectl get pods --show-labels
NAME                            READY   STATUS        RESTARTS   AGE     LABELS
aceest-blue-67d5776b67-5d4kv    1/1     Running       0          10m     app=aceest,pod-template-hash=67d5776b67,version=blue
aceest-blue-67d5776b67-6ksx8    1/1     Running       0          11s     app=aceest,pod-template-hash=67d5776b67,version=blue
aceest-blue-67d5776b67-g42h2    1/1     Running       0          10m     app=aceest,pod-template-hash=67d5776b67,version=blue
aceest-green-64fc4ffdf5-fmqt2   1/1     Running       0          7m21s   app=aceest,pod-template-hash=64fc4ffdf5,version=green
aceest-green-64fc4ffdf5-j6m2g   1/1     Terminating   0          7m21s   app=aceest,pod-template-hash=64fc4ffdf5,version=green
mkayfour@192 aceest-fitness-gym %
```

Traffic Routing Mechanism

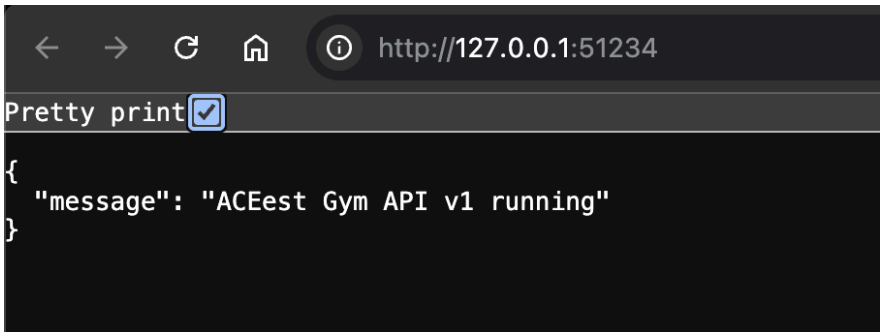
A single Kubernetes Service is used:
yaml selector: app: aceest version: blue

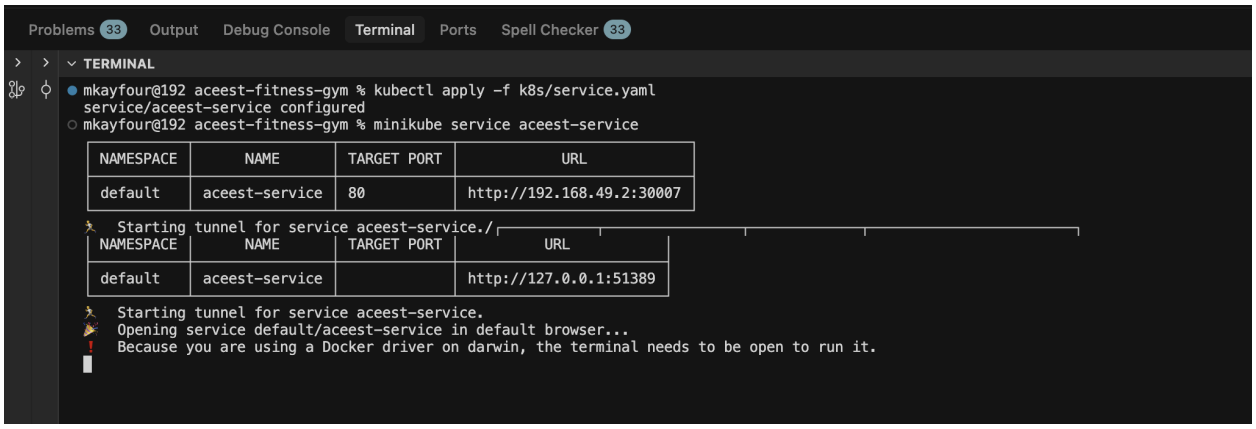
Switching Traffic

- To deploy new version:
 bash kubectl patch service aceest-service \ -p
'{"spec":{"selector":{"app":"aceest","version":"green"}}}'

- Rollback:
 bash kubectl patch service aceest-service \ -p
'{"spec":{"selector":{"app":"aceest","version":"blue"}}}'

Deployment on local





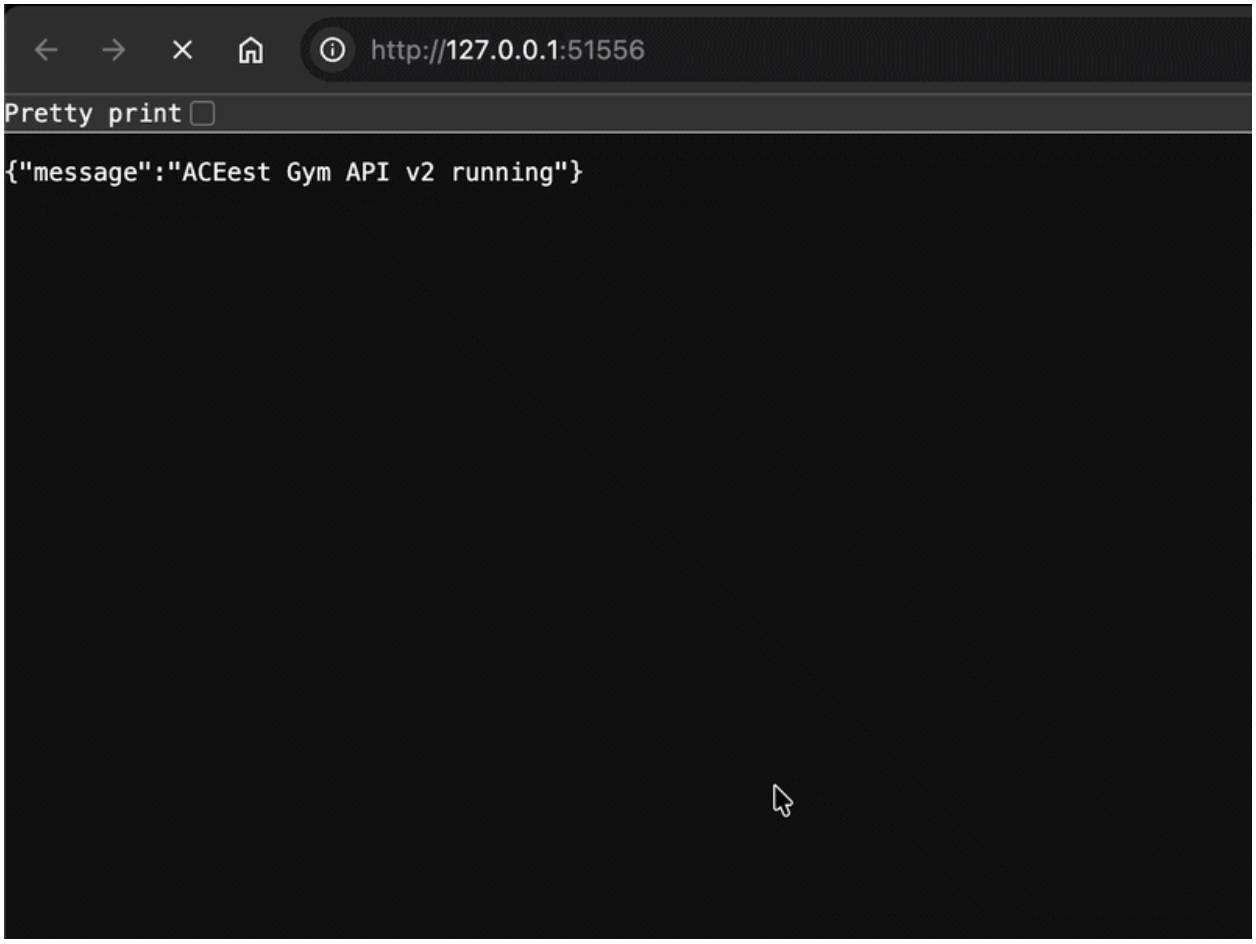
Blue-green deployment on local

Blue-green deployment ensures zero downtime by maintaining two environments and switching

traffic only after validation. This reduces deployment risk and enables instant rollback.

```
mkayfour@192 aceest-fitness-gym % kubectl get endpoints aceest-service
NAME             ENDPOINTS                                                     AGE
aceest-service   10.244.0.19:5000,10.244.0.20:5000,10.244.0.21:5000 + 1 more...   88m
mkayfour@192 aceest-fitness-gym % kubectl scale deployment aceest-blue --replicas=3
deployment.apps/aceest-blue scaled
mkayfour@192 aceest-fitness-gym % kubectl scale deployment aceest-green --replicas=1
deployment.apps/aceest-green scaled
mkayfour@192 aceest-fitness-gym % kubectl get pods --show-labels
NAME                            READY   STATUS        RESTARTS   AGE     LABELS
aceest-blue-67d5776b67-5d4kv    1/1     Running       0          10m     app=aceest,pod-template-hash=67d5776b67,version=blue
aceest-blue-67d5776b67-6ksx8    1/1     Running       0          11s     app=aceest,pod-template-hash=67d5776b67,version=blue
aceest-blue-67d5776b67-g42h2    1/1     Running       0          10m     app=aceest,pod-template-hash=67d5776b67,version=blue
aceest-green-64fc4ffdf5-fmqt2   1/1     Running       0          7m21s   app=aceest,pod-template-hash=64fc4ffdf5,version=green
aceest-green-64fc4ffdf5-j6m2g   1/1     Terminating   0          7m21s   app=aceest,pod-template-hash=64fc4ffdf5,version=green
mkayfour@192 aceest-fitness-gym %
```

Canary deployment was simulated by running two versions (v1 and v2) simultaneously. Traffic was distributed between versions to validate the new release before full rollout.



Key Challenges & Solutions

1. Architecture Mismatch (Critical)
- Issue: exec format error
- Cause: ARM image built on Mac (M1/M2)

- Fix:
  docker buildx build --platform linux/amd64

2. Docker Socket Permission Issue
- Issue: Jenkins unable to access Docker daemon
- Fix:
  - Mounted /var/run/docker.sock
  - Adjusted permissions

3. Kubernetes Authentication Issues
- Issue: kubectl failing inside container
- Fix:
  - Mounted .kube and .minikube configs
  - Installed gke-gcloud-auth-plugin

4. Invalid Image Name
- Issue: InvalidImageName
- Cause: Placeholder (YOUR_PROJECT_ID) not replaced
- Fix: Use actual project ID in YAML

5. CrashLoopBackOff Errors
- Root causes:
  - Wrong image architecture
  - Incorrect container startup
- Fix:
  - Rebuilt image properly
  - Ensured correct runtime command

Reliability Enhancements

Health Checks
Added:
yaml readinessProbe: httpGet: path: / port: 5000 livenessProbe: httpGet: path: / port: 5000

Benefits:
- Prevents unhealthy pods from receiving traffic
- Enables automatic recovery

Resource Management
yaml resources: requests: cpu: "100m" memory: "128Mi" limits: cpu: "500m" memory: "512Mi"

Auto Scaling
bash kubectl autoscale deployment aceest-blue --cpu-percent=70 --min=2 --max=5

Key Automation Outcomes

- Automated build and deployment using Jenkins
- Docker image versioning and registry integration
- Kubernetes-based scalable deployment
- Blue-Green deployment ensuring zero downtime
- Ability to rollback instantly by switching service selector
- Health monitoring using readiness and liveness probes

Single deployment

```
TERMINAL
(venv) mkayfour@192 aceest-fitness-gym % kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
aceest-app-859c56599-9l24g    1/1     Running   0          2m16s
aceest-app-859c56599-th8zs    1/1     Running   0          2m16s
(venv) mkayfour@192 aceest-fitness-gym % kubectl get svc
NAME             TYPE           CLUSTER-IP       EXTERNAL-IP      PORT(S)        AGE
aceest-service   LoadBalancer   34.118.239.209   35.200.230.182   80:31881/TCP   8m57s
kubernetes       ClusterIP      34.118.224.1     <none>           443/TCP        16m
(venv) mkayfour@192 aceest-fitness-gym %
```

Blue green deployment on Google Cloud

```
Problems 34   Output   Debug Console   Terminal   Ports   Spell Checker 34
TERMINAL
(venv) mkayfour@192 aceest-fitness-gym % kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
aceest-blue-7469dd969-9lqnw   1/1     Running   0          2m11s
aceest-blue-7469dd969-c2vjd   1/1     Running   0          2m10s
aceest-green-66ddbc5856-4pqk9 1/1     Running   0          2m8s
aceest-green-66ddbc5856-bht7k 1/1     Running   0          2m11s
(venv) mkayfour@192 aceest-fitness-gym %
```

Switching between blue green deployments and rollback

```
Problems 34   Output   Debug Console   Terminal   Ports   Spell Checker 34
TERMINAL
(venv) mkayfour@192 aceest-fitness-gym % kubectl get endpoints aceest-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME             ENDPOINTS                          AGE
aceest-service   10.16.0.26:5000,10.16.1.27:5000   16m
(venv) mkayfour@192 aceest-fitness-gym % kubectl patch service aceest-service \
  -p '{"spec":{"selector":{"app":"aceest","version":"green"}}}'
service/aceest-service patched
(venv) mkayfour@192 aceest-fitness-gym % kubectl get endpoints aceest-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME             ENDPOINTS                          AGE
aceest-service   10.16.0.27:5000,10.16.1.28:5000   16m
(venv) mkayfour@192 aceest-fitness-gym % kubectl patch service aceest-service \
  -p '{"spec":{"selector":{"app":"aceest","version":"blue"}}}'
service/aceest-service patched
(venv) mkayfour@192 aceest-fitness-gym % kubectl get endpoints aceest-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME             ENDPOINTS                          AGE
aceest-service   10.16.0.26:5000,10.16.1.27:5000   16m
(venv) mkayfour@192 aceest-fitness-gym %
```

Final deployment on google cloud

```
Not Secure   35.200.230.182
Pretty print
{"message":"ACEest Gym API running by Mohit Kumar (2024TM93706)"}
```

Code Quality & Testing

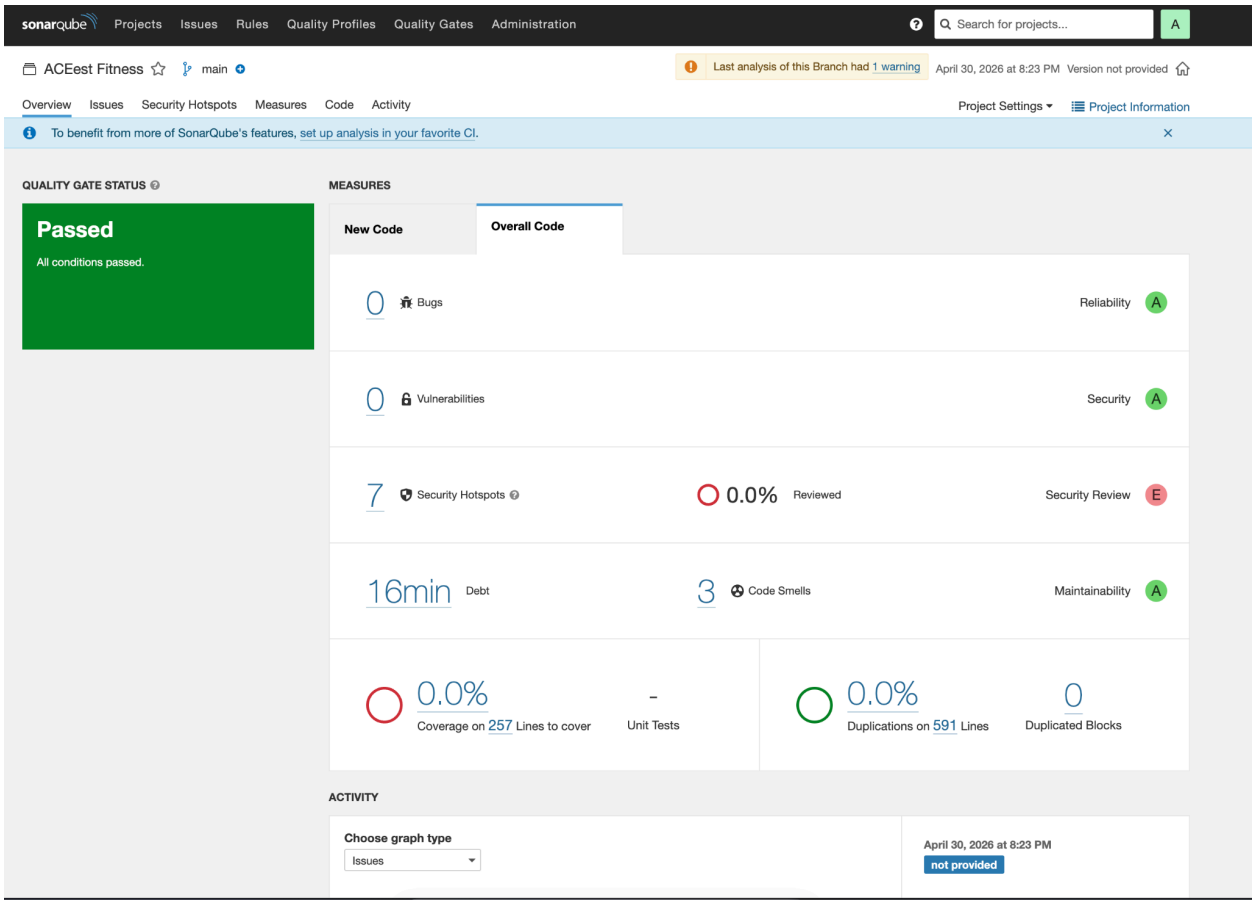
[PYTEST]

```
TERMINAL
(venv) mkayfour@192 aceest-fitness-gym % pytest
========================================================================================== test session starts ==========
platform darwin -- Python 3.10.4, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/mkayfour/workspace/aceest-fitness-gym
configfile: pytest.ini
collected 17 items

tests/test_app.py .................

========================================================================================== 17 passed in 0.14s ==========
(venv) mkayfour@192 aceest-fitness-gym %
```

[SONARQUBE]



Conclusion

This project demonstrates a complete end-to-end cloud deployment pipeline using modern DevOps practices. It highlights real-world challenges such as architecture mismatches, container orchestration issues, and deployment strategies, and provides practical solutions to address them.

This project provided hands-on experience with real-world DevOps challenges including container compatibility, Kubernetes debugging, and deployment strategies. The successful implementation demonstrates practical understanding of CI/CD pipelines and cloud-native deployment practices.